Got it — here's a complete **`README.md`** you can paste straight into `rev-performance-ZNC07/README.md`. It includes a Quick Start, the knobs, and the analyst-facing outputs, all in one place.

Revenue Performance Pipeline (rev-performance-ZNC07)

End-to-end Python pipeline to:

- Clean and enhance invoice-level data (including canonical **NRV Gap ($/%/Sum)** and benchmark wiring).
- Produce **weekly granular + aggregated** insights with **leakage-free**, **adaptive** benchmarks (median / winsorized / mean with EWMA + shrinkage options).
- (Optional) Generate **NRV gap benchmarks** at two grains and merge them into weekly outputs.

Quick Start (Git, venv, install, run)

```
# 1) Clone your repo
git clone https://github.com/<your-org-or-user>/rev-performance-ZNC07.git
cd rev-performance-ZNC07

# 2) (Recommended) Create a clean virtual environment
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate

# 3) Install dependencies
pip install -r requirements.txt

# 4) (Optional) Set knobs (env vars) – see "Configuration Knobs" section for details
export DATA_DIR="/mnt/data"
export EXCLUDE_LATEST_FROM_BENCH="1"
export BACKTEST_K="6"

# 5) Run the pipeline (core)
```

python 01_preprocess_and_enhance.py
python 02_weekly_insights.py

6) (Recommended optional) NRV gap benchmarks + merge back into weekly
python 03_nrv_gap_benchmarks.py
python 03b_merge_nrv_gaps_into_weekly.py

Where is my data?
By default the pipeline reads/writes under `DATA_DIR` (default: `/mnt/data`). You can change it via `export DATA_DIR="/path/to/your/data"` before running.

Files & Responsibilities

01_preprocess_and_enhance.py

Purpose: Read raw invoice-level extract, remove "total" rows, normalize column names, **guarantee NRV Gap ($), NRV Gap (%), NRV Gap Sum ($)**, compute per-invoice benchmarks (mean/median/winsorized), and export the enhanced file + underpayment summary.

Inputs (expects in `DATA_DIR`):

- `preprocess_invoice_data_output.xlsx` (preferred)
  or `preprocess_invoice_data_output.csv`
  (Fallbacks allowed: `invoice_level_index_enhanced.xlsx`, `RMT Invoice_level_index.xlsx`.)

Primary outputs:

- `enchance_invoice_metrics_output.csv`
- `enchance_invoice_metrics_output.zip` (contains the CSV)
- `enchance_invoice_metrics_underpayment_summary.csv`
- `enchance_invoice_metrics_underpayment_summary.xlsx` (with header comments & data dictionary)

Key guarantees / behavior:

- Canonical NRV columns always present (renamed or created as NaN with a log note): `NRV Gap ($)`, `NRV Gap (%)`, `NRV Gap Sum ($)`

- Underpayment subsets include:
    - OpenInv == 1 & Insurance>0 (unchanged)
    - **OpenInv == 0 (Insurance filter removed — updated)**
    - OpenInv == 1 & Insurance>0 & AR90>0

- Underpayment vs **85% E/M** and vs **benchmarks (mean/median/winsorized)** computed **at invoice level** to avoid CPT-line double counting.

02_weekly_insights.py

Purpose: Turn invoice lines into weekly granular + aggregated outputs with **leakage-free**, **adaptive** benchmark selection per `Benchmark_Key`. Persist full backtest diagnostics and per-invoice totals for external comparisons.

Benchmark strategy:
Adaptive per-key selection among **median / winsorized / mean** (EWMA + shrunk median are candidates in backtest diagnostics). Selection uses **SMAPE** by default (or **WMAPE** if enabled), with **MASE** tie-break on volatile keys. History built strictly prior to target weeks (no look-ahead), with Hampel and adaptive winsorization to stabilize outliers.

Inputs (expects in `DATA_DIR`):

- Enhanced invoice-level file: `enchance_invoice_metrics_output.csv`

- (Optional) Underpayment summary: `enchance_invoice_metrics_underpayment_summary.(csv|xlsx)` — auto-merged if present.

Primary outputs:

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular.csv`

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular.zip`
  (includes: granular CSV **+** `invoice_totals_backtest.csv` **+** selection diagnostics)
- `v2_Rev_Perf_Weekly_Model_Output_Final_agg.csv`
- `v2_Rev_Perf_Weekly_Model_Output_Final_agg.xlsx`
- Diagnostics:
  - `v2_Rev_Perf_Weekly_Model_Backtest_Selection.csv`
  - `v2_Rev_Perf_Weekly_Model_Backtest_Selection.xlsx`
  - `invoice_totals_backtest.csv`

Grain:
Weekly groupings by **Year | Week | Payer | Group_EM | Group_EM2 | Benchmark_Key**.

03_nrv_gap_benchmarks.py *(optional but recommended)*

Purpose: Compute **NRV mismatch** benchmarks where `Payment Amount* != NRV Zero Balance*` at **two grains**:

1. **Invoice + CPT grouping:**
   `Year | Week | Payer | Group_EM | Group_EM2 | CPT_List_Str | Invoice_Number`
2. **CPT grouping only:**
   `Year | Week | Payer | Group_EM | Group_EM2 | CPT_List_Str`

For each grain, produce totals, counts, mean/median/winsorized gaps, and simple rates (share of mismatched invoices). These help pinpoint operational gaps with CPT context.

Inputs:

- `enchance_invoice_metrics_output.csv` (from Step 01)

Primary outputs:

- `nrv_gap_invoice_week.csv` (invoice + CPT grain)
- `nrv_gap_week.csv` (CPT-only weekly grain)

03b_merge_nrv_gaps_into_weekly.py

Purpose: Merge NRV gap benchmark outputs back into the weekly outputs from Step 02 (left-join by compatible keys), so analysts and dashboards see both **adaptive payment benchmarks** and **NRV mismatch hotspots** in one place.

Inputs:

- Step 02 weekly outputs (granular and/or agg)
- Step 03 outputs: `nrv_gap_invoice_week.csv`, `nrv_gap_week.csv`

Primary outputs:

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular_with_nrv.csv`
- `v2_Rev_Perf_Weekly_Model_Output_Final_agg_with_nrv.csv`

Configuration Knobs (Environment Variables)

> Set these before running (e.g., `export NAME=value`). Sensible defaults are embedded, so you can run without setting anything.

Common

- `DATA_DIR` (default: `/mnt/data`)
  Working directory for input/output files.

Step 01 — preprocess & enhance

- *(No mandatory knobs beyond `DATA_DIR`.)*
  Script auto-detects `preprocess_invoice_data_output.xlsx` or CSV in `DATA_DIR`.
  It will **canonicalize NRV columns** and print what it changed/created.

Step 02 — weekly insights & adaptive selection

- `EXCLUDE_LATEST_FROM_BENCH` (default: `1`)
  If `1`, exclude the latest **(Year, Week)** from historical window to prevent leakage.

- `BACKTEST_K` (default: `6`)
  Number of target weeks to roll during backtest per key.

- `WMAPE_SELECTION` (default: `0`)
  If `1`, use **WMAPE** instead of **SMAPE** as the primary selection metric.

- `N_MIN_INVOICES` (default: `40`)
  Adaptive historical window grows until at least this many invoices are included.

- `SHRINK_LAMBDA` (default: `20`)
  Prior strength (in invoice units) for shrinkage of the per-invoice median toward **payer×EM prior**.

- `EWMA_ALPHA` (default: `0.2`)
  Smoothing for EWMA candidate over weekly per-invoice medians.

- `HAMPEL_K` (default: `3.0`)
  Hampel filter threshold (in MAD units) to drop week-level outliers from history.

- **Winsorization bounds**
  - `WINSOR_L` / `WINSOR_U` (defaults: `0.05` / `0.95`) for N≥20
  - `WINSOR_L_SMALL` / `WINSOR_U_SMALL` (defaults: `0.10` / `0.90`) for small-N

Step 03 — NRV gap benchmarks *(optional)*

- `NRV_MISMATCH_TOLERANCE` (default: `0`)
  Treat `|Payment - NRV Zero Balance| <= tolerance` as **match** (not a

mismatch).

Step 03b — merge NRV gaps *(optional)*

- *(No special knobs; uses `DATA_DIR` and merges on compatible keys.)*

Inputs (Required vs Optional)

Required to run core pipeline (Steps 01–02):

- `DATA_DIR/preprocess_invoice_data_output.xlsx` *(preferred)*
  or `DATA_DIR/preprocess_invoice_data_output.csv`

Optional but recommended:

- `DATA_DIR/enchance_invoice_metrics_underpayment_summary.(csv|xlsx)`
  (If present, Step 02 will auto-merge extra underpayment columns.)

Optional for NRV gap add-on (Steps 03–03b):

- None beyond the outputs of Step 01 and Step 02.

Outputs (Analyst Facing)

After running **Steps 01–02**, you'll have:

- `enchance_invoice_metrics_output.csv`
  Row-level enhanced extract with **guaranteed** NRV Gap columns, per-invoice benchmark stats (mean/median/winsorized), and readiness for weekly modeling.

- `enchance_invoice_metrics_underpayment_summary.(csv|xlsx)`
  Invoice-level underpayment rollups:

    - vs. **85% E/M** (rate vs collection split, κ scaffolds for Payer/Key, adjusted totals)
    - vs. **benchmarks** (mean/median/winsorized)
    - Subsets (OpenInv==1 & Insurance>0, **OpenInv==0**, OpenInv==1 & Insurance>0 & AR90>0)

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular.csv`
  Weekly **granular** insights per `Year|Week|Payer|Group_EM|Group_EM2|Benchmark_Key` with **Selected_Benchmark_Payment** and variance vs selected benchmark, plus (if present) merged underpayment summary columns.

- `v2_Rev_Perf_Weekly_Model_Output_Final_agg.(csv|xlsx)`
  Same grain, aggregation-friendly version (wide measurements).

- Diagnostics:
    - `v2_Rev_Perf_Weekly_Model_Backtest_Selection.(csv|xlsx)` — per-key backtest metrics (MAE, MAPE, **SMAPE**, **WMAPE**, **MASE**) and **Benchmark_Method_Selected**.
    - `invoice_totals_backtest.csv` — per-invoice totals per week (enables external method comparisons).

If you also run **Steps 03–03b**, you'll get:

- `nrv_gap_invoice_week.csv`
  NRV mismatch stats at: `Year|Week|Payer|Group_EM|Group_EM2|CPT_List_Str|Invoice_Number`.

- `nrv_gap_week.csv`
  NRV mismatch stats at: `Year|Week|Payer|Group_EM|Group_EM2|CPT_List_Str`.

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular_with_nrv.csv`
  Weekly granular outputs enriched with NRV mismatch metrics.

- `v2_Rev_Perf_Weekly_Model_Output_Final_agg_with_nrv.csv`
  Aggregated outputs enriched with NRV mismatch metrics.

How the Benchmarks Work (Step 02)

- We build a **history strictly prior** to each target week (no leakage).
- We evaluate candidate methods on **per-invoice** values:
    - **Median**, **Winsorized mean**, **Mean**
    - EWMA (on weekly medians) + **shrunk median** (toward **payer×EM prior**) in diagnostics
- We scale the per-invoice estimate by the **current week's visit count** and compute **SMAPE** (or **WMAPE**, if enabled), with **MASE** tie-break on volatile keys.
- The **winner** becomes `Benchmark_Method_Selected` for that `Benchmark_Key`.

This is separate from **85% E/M** — 85EM is still exported and used in the underpayment summary (Step 01), but **does not** drive the adaptive benchmark selection in Step 02.

Typical Runbook

1. **Prepare inputs**
   Place `preprocess_invoice_data_output.xlsx` in `DATA_DIR` (default `/mnt/data`).

Run Step 01

```
python 01_preprocess_and_enhance.py
```

2. Check console logs for NRV canonicalization notes and "Rows removed as totals".

Run Step 02

```
python 02_weekly_insights.py
```

3. Verify creation of:

- `v2_Rev_Perf_Weekly_Model_Output_Final_granular.csv`
- `v2_Rev_Perf_Weekly_Model_Backtest_Selection.csv`

(Optional) Run NRV gap benchmarks

```
 python 03_nrv_gap_benchmarks.py
python 03b_merge_nrv_gaps_into_weekly.py
```

4. Verify `_with_nrv.csv` outputs.

Troubleshooting

- **"Missing file"**: Ensure your input file is under `DATA_DIR` and named correctly.
- **No NRV columns**: Step 01 will create them as NaN if not found; check the log for renames.
- **All benchmark underpay fields are zero** (Step 01 summary): Usually a **key mismatch** → verify `Benchmark_Key` formation and that `Benchmark_Avg_Payment_temp*` columns were created.
- **Selection looks unstable**: Consider increasing `N_MIN_INVOICES`, increasing `SHRINK_LAMBDA`, or enabling `WMAPE_SELECTION=1`.

Repo Layout (suggested)

```
rev-performance-ZNC07/
├── 01_preprocess_and_enhance.py
├── 02_weekly_insights.py
├── 03_nrv_gap_benchmarks.py          # optional
├── 03b_merge_nrv_gaps_into_weekly.py   # optional
├── requirements.txt
└── README.md
```

If you want a one-shot runner later, we can add a `master_pipeline.py` wrapper — but per your latest design, the four scripts above are sufficient.

If you want, I can also generate a tiny **`.env.example`** with the most important knobs pre-filled.